

```

    local filter: Log::Filter = [$name="split-mod5s", $path_
func=Factor::mod5];
    Log::add_filter(Factor::LOG, filter);
    Log::remove_filter(Factor::LOG, "default");
}

event bro_done()
{
    local numbers: vector of count = vector(1, 2, 3, 4, 5, 6,
7, 8, 9, 10);
    for ( i in numbers)
    {
        local result = Factor::factorial(numbers[i]);
        Log::write(Factor::LOG, [$num=numbers[i],
                                $factorial_num=result]);
        if (result == Factor::interesting_result)
        {
            NOTICE([$note=Factor::Interesting_Result,
$msg = "Something happened!",
                $sub = fmt("result = %d", result)]);
        }
    }
}

```

The examples illustrate the core scripting features of the Zeek script. Though the functionality coded is simple, the aim was to showcase the abilities of Zeek such as writing logs, scanning and raising notices for interesting events.

Zeek frameworks

Zeek provides many frameworks. These enable us to perform specific tasks easily and effectively. The Zeek frameworks are listed below:

- Configuration framework
- File analysis
- Geo-location
- Input framework
- Logging framework
- NetControl framework
- Notice framework
- Signature framework
- Summary statistics
- Cluster framework

Covering all these frameworks is not possible due to space constraints. The Intel framework that is used along with Zeek can be used to perform interesting tasks. One such example is shown below:

```

@load frameworks/intel/seen

redef Intel::read_files += {
    fmt("%s/intel-1.dat", @DIR)
};

```

Output Logs

capture_loss	conn	files	http	intel	known_hosts	known_services	software
stats							
ts	host	port_num	port_proto	service			
1320279566.452687	132.235.215.119	80	tcp	HTTP			
1320279566.831473	72.21.211.173	80	tcp	HTTP			
1320279567.211031	184.72.234.3	80	tcp	HTTP			
1320279567.211407	174.129.249.33	80	tcp	HTTP			
1320279567.536586	74.125.225.78	80	tcp	HTTP			
1320279567.690049	74.125.225.91	80	tcp	HTTP			
1320279571.625521	69.171.228.39	80	tcp	HTTP			
1320279571.883692	132.235.215.117	80	tcp	HTTP			
1320279577.475501	216.34.181.48	80	tcp	HTTP			

Figure 5: Zeek Intel output

```
};
```

ntel-1.dat consists of the following:

```
www.reddit.com Intel::DOMAIN my_special_source
```

This simple code generates a detailed output. The output includes the following components:

- *Capture_loss*
- *Connection*
- *Files*
- *Http*
- *Intel*
- *Known_Hosts*
- *Known_Services*
- *Software*
- *Stats*

Detailed instructions on the intelligent framework are available at <https://docs.zeek.org/en/stable/frameworks/intel.html>.

One of the major advantages of Zeek is that it comes with comprehensive documentation. Detailed teaching and training modules are available at <https://www.zeek.org/playground/index.html>. Zeek videos are available at <https://www.youtube.com/zeekurity>.

This article is only a sneak peek into the world of Zeek. With the resources listed above, interested readers can easily explore it. **END** 🐧

References

- [1] Zeek homepage: <https://www.zeek.org/>
- [2] Zeek documentation: <https://www.zeek.org/documentation/index.html>

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science at the Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry. He can be reached via mail at kskuppu@gmail.com.